Repository Investigation and Reproducible Build

ISE A3 | Intelligent Software Engineering | 25 October 2026

Project context: Assignment Deadline Change Tracking and Verification System

Purpose and principal finding

This investigation examines Todoman, an unfamiliar personal task repository, as a possible storage and presentation boundary for the deadline-change workflow established in A1 and specified in A2. Ordinary due-date editing is reproducible. Academic verification, stale-version protection and explicit-instant semantics require a controlled boundary beyond the examined CLI write contract.

The study freezes an upstream commit, builds its source in an isolated environment, reconstructs one executed deadline edit, and compares observed behavior with A2. It retains negative results and distinguishes facts, inferences and proposed changes. Upstream source remains unchanged.

Frozen repository and scope

Item	Recorded identity
Repository	https://github.com/pimutils/todoman
Commit	78e35788606a0add0c4f99ceb702c3ea0d0b1346
Snapshot label	0.0+g78e35788606a; study label, not a released version
Licence and input	ISC; complete source export and synthetic VTODO fixture retained
Execution target	CPython 3.12.14; Linux x86_64; UTC; offline locked wheels

Recorded outcomes

Evidence	Outcome	Interpretation
Upstream cases	197 total	188 passed; 6 skipped; 3 expected failures
Study checks	6 observations passed	Includes observed missing safeguards
Clean build and start	Successful	Wheel built, installed and CLI started offline
Future integration	Proposed only	Verification service and export adapter are not implemented

All task records are synthetic. No real student information, live university source, institutional credentials or LMS connection was used. A1 simulated pilot data remain simulated.

Reproduction method and environment

The complete offline procedure is scripts/reproduce.py. It verifies 33 input hashes, creates a new virtual environment, installs 30 pinned dependency distributions, builds and installs the exported source, executes tests, starts the CLI, runs the probes and checks dependencies. Each run writes a new directory and retains stage commands, exit codes and raw output.

```
python3 --version
python3 scripts/reproduce.py --verify-only
python3 scripts/reproduce.py --output runs/local
```

Environment and build identity

The runtime record identifies CPython 3.12.14, Linux x86_64, glibc 2.39 and the SQLite version. TZ=UTC, LC_ALL=C.UTF-8, PYTHONHASHSEED=0, DETERMINISTIC_TESTS=true and Hypothesis seed 20261025 are fixed. Each dependency wheel is hash-locked. Native Windows is excluded by upstream; WSL/Linux guidance is provided but WSL was not separately tested.

A Git export contains no SCM metadata. The build therefore supplies SETUPTOOLS_SCM_PRETEND_VERSION_FOR_TODOMAN with the documented snapshot label. SOURCE_DATE_EPOCH is derived from the pinned commit timestamp. Two clean builds produced the same recorded wheel hash. This does not establish reproducibility on every platform.

Retained test exclusions

Cases	Reason	Treatment
1 skip	Optional click_repl is absent	Optional REPL is not tested
4 skips	Invalid recurrence combinations	Preserved as upstream exclusions
1 skip	UI case linked to issue 537	No interactive UI guarantee
3 expected failures	Issue 9 and two unimplemented config cases	Retained, never counted as passes

The suite reports 16 warnings, principally dependency API deprecations. Default coverage flags are explicitly disabled; no coverage percentage is claimed. Raw XML and text preserve every test and reason.

Failed and nondeterministic evidence

The first study harness failed because an icalendar list was passed directly to next(). That investigator error is retained and corrected with iter(...). Malformed input and missing configuration intentionally produce nonzero exit codes. Absolute due values and UID are fixed; LAST-MODIFIED, mtimes, paths and timings vary and are not byte-identity oracles.

Architecture and dependency boundaries

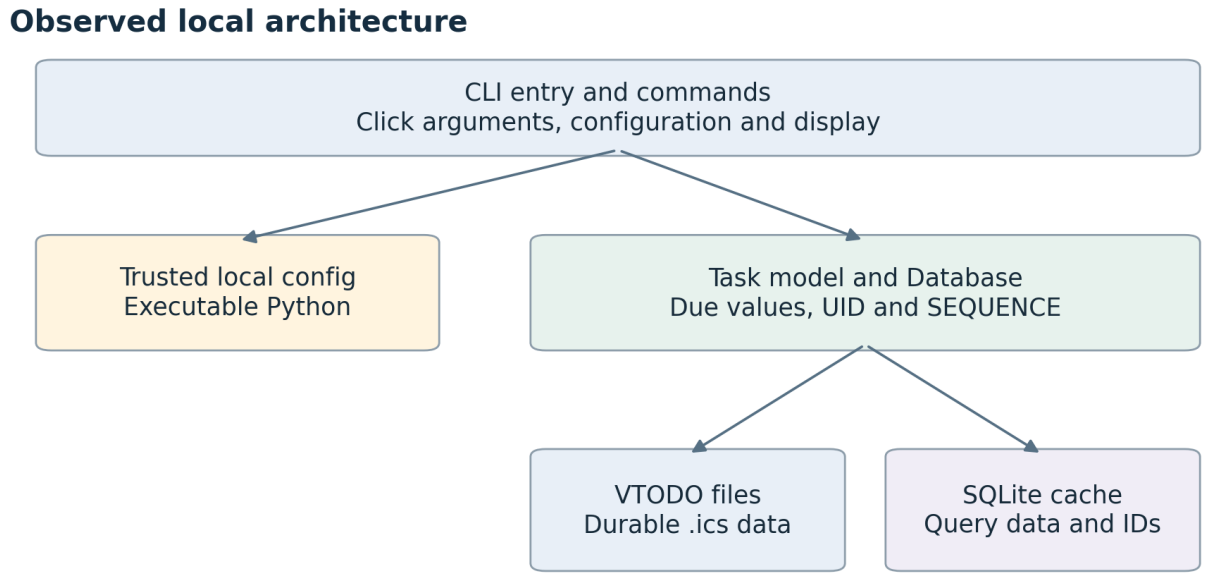


Figure 1. Observed local architecture. Lines represent examined dependencies and calls; no external network synchronization was executed.

Component	Source location	Role
Entry and commands	__main__.py; cli.py	CLI startup, validation and edit/show/list commands
Configuration	configuration.py:220	Executes trusted local config and validates settings
Dates and feedback	formatters.py:255,262	Parses configured local dates/times and formats output
Persistence	model.py:1064,1164	Database facade writes VTODO and refreshes cache
Files and cache	model.py:296,422	Durable ICS records and derived SQLite query data

Critical dependencies are Click/click-log, icalendar, python-dateutil, parsedatetime, pyxdg, humanize and urwid. Build and test tools and all transitive installed distributions are locked separately from standard-library sqlite3. Urwid widgets were inspected but not interactively exercised.

Interfaces include CLI arguments, JSON porcelain output, local files and optional editor integration. Separate synchronization software may consume calendar files; no live CalDAV or LMS integration was tested. SQLite also assigns local task IDs, which must not be assumed to be institutional assignment identifiers.

Executed deadline edit and evidence chain

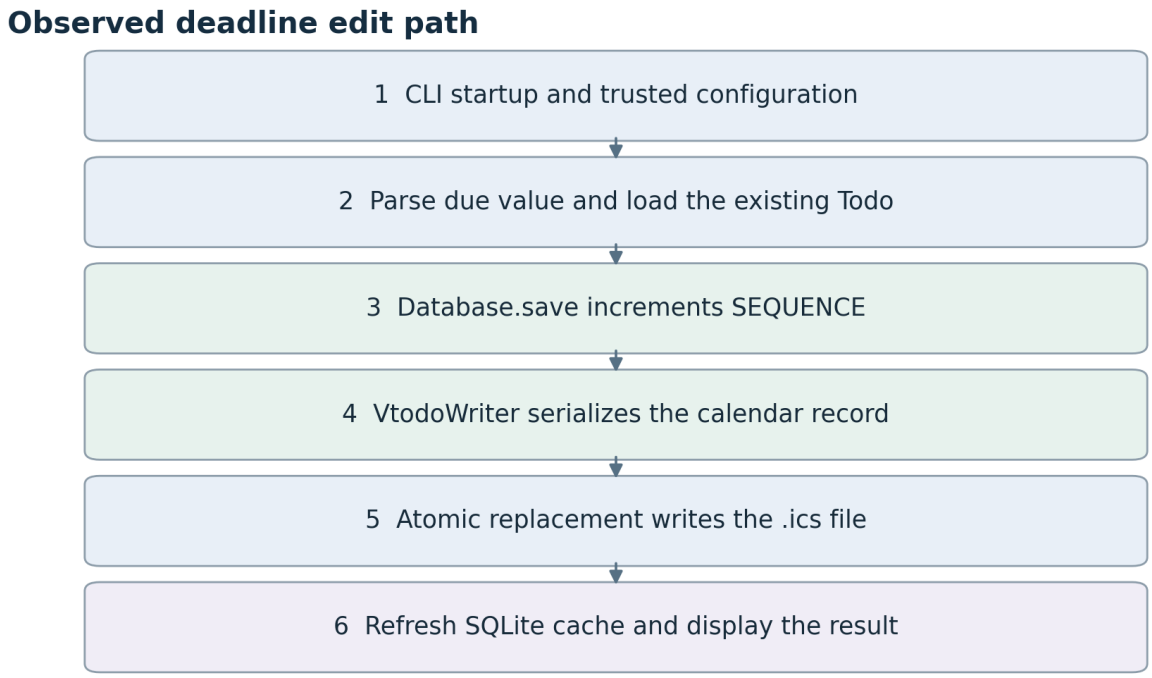


Figure 2. Reconstructed path supported by the selected runtime trace, source definitions and persisted data.

P1 begins with a fixed synthetic UID, DUE 10 October 2026 at 16:30 UTC and SEQUENCE 1. An edit proposes 17 October at 16:30 UTC. The resulting ICS file stores that instant and SEQUENCE 2; a fresh CLI process and direct SQLite observation agree. This UTC fixture is separate from the Asia/Shanghai A2 examples.

The profiler wrapper invokes the real todoman.__main__ module without changing application source. Its 28 selected calls include configuration, date validation, task retrieval, Database.save, VtodoWriter, atomic_write, cache refresh/commit and feedback. It is a filtered call trace, not a complete coverage result.

Classification	Supported statement
Runtime fact	Normal edit, file replacement, cache row and fresh-process readback were observed.
Source fact	File replacement precedes a separate SQLite cache commit.
Inference	An interruption between those operations could create temporary divergence; not fault-injected.
Proposal	A separate service would own verified academic records and export approved copies.

Characterization checks and counterexamples

The six checks inspect files, cache rows, actual exit codes or a harmless execution marker. A passing check means the stated behavior was observed, including behaviors that fail the A2 contract. It does not establish academic or production security.

ID	Procedure	Observed result and limit
P1	Normal deadline edit	Due value persists; SEQUENCE increases from 1 to 2; a new CLI process and SQLite cache agree. No institutional source or owner verification is established.
P2	Malformed date	Unrecognized input exits 2; the existing ICS file remains byte-identical. This covers one malformed date, not an exhaustive parser study.
P3	Stale object saves	Two Todo objects are read before either save. Saving both accepts the stale replacement and repeats the resulting sequence. This is a deterministic interleaving, not a measured parallel race.
P4	Date-only deadline	The CLI accepts 25 October 2026 and persists an all-day date. That is legitimate task behavior but lacks the exact cutoff instant required by A2.
P5	Executable configuration	Loading a synthetic Python config writes an isolated harmless marker. This confirms that config belongs within a trusted local deployment boundary.
P6	Missing configuration	An explicitly nonexistent config exits 1 with actionable feedback. No real account or academic record is involved.

Evidence retained

evidence/verified/workflow contains command stdout/stderr, results.json, call-trace.json, original and revised calendar snapshots, and cache-after-edit.json. The upstream JUnit XML preserves expected failures and exclusions. The failed first harness attempt remains separately identified as an investigator error.

Relationship to the earlier assignments

A1 defines the synchronization problem; its simulated pilot does not prove prevalence. A2 defines the desired owner, source, version, rejection and time rules. Todoman has a different personal-task contract. The observed gaps identify an adaptation boundary rather than automatically constituting defects in upstream software.

Three risks and acceptance conditions

Risk priority is qualitative and scoped to reusing the repository within the academic workflow. No probability, incident frequency or CVSS score is fabricated.

R1 Direct writes lack the A2 verification and stale-version boundary

The CLI/model is reused as the authoritative academic-deadline writer or multiple stale clients save the same record. Source: cli.py edit; model.py Database.save. Runtime: P3 overwrites a newer due value and repeats the resulting sequence. An unreviewed or stale proposed deadline can replace a personal record without the A2 owner/source/version contract.

Response: Use a protected A2 service as record owner and export only approved revisions; compare alternatives in ADR-001. Acceptance: Non-owner/unverified/stale updates rejected; one revision per confirmation; retry creates no duplicate history.

Residual limit: Direct external writers can still change exported calendars; export is not academic authority.

R2 Date-only and local-time semantics differ from the A2 explicit-instant contract

A date or locally interpreted value is mapped to an academic deadline without explicit time and institutional zone. Source: formatters.py parse_datetime; model.py normalize_datetime. Runtime: P4 persists a date-only DUE value. The system could display an all-day date where an exact cutoff instant is required.

Response: Validate explicit timestamp, offset and named zone before export; preserve the original source zone in the owning service. Acceptance: Missing time/zone and offset mismatch rejected; UTC and zone round trips checked against A2 examples.

Residual limit: Ambiguous institutional notices require human clarification; no DST study was executed.

R3 Executable Python configuration crosses a trust boundary if accepted as uploaded data

A future web/API workflow accepts user-provided Todoman config files or builds config from unchecked text. Source: configuration.py load_config uses exec_module. Runtime: P5 writes an isolated harmless marker. Configuration code runs with process permissions; this is inappropriate for untrusted notices or uploads.

Response: Keep deployment-owned configuration fixed; accept only validated data in the future service and never load uploaded Python. Acceptance: Uploaded config/code is not accepted or executed; service account has limited filesystem access.

Residual limit: A trusted deployment administrator can still run local code; sandbox enforcement was not implemented.

Owners are future implementation/deployment roles, not named reviewers or assigned individuals. The complete register, including conditions, priorities, ownership and residual limits, is supplied in Markdown, CSV and JSON.

Architecture decision and alternatives

ADR 001 addresses R1: how to add the A2 verification and version boundary while preserving a reproducible upstream baseline. Status: proposed for future implementation; A3 makes no product change.

Criterion	A Modify upstream write core	B Separate service and export
Rule ownership	Add academic concepts to CLI/model	Keep A2 rules in a protected service
Change surface	Reconcile command, model, cache and other writers	Add a bounded service and export adapter
Version safety	Retrofit coordinated conflict checks/history	Transactional service record; export after commit
Existing writers	Every write path must adopt the rules	External files remain derived, unverified copies
Main cost	Fork maintenance and compatibility	New identity mapping, store and recovery behavior

Proposed decision

Prefer B. A future service would authenticate ownership, verify source authority independently, validate candidate state and explicit time semantics, compare the expected version and commit one history entry before export. It would own the personal verified record, not the official university deadline. Todoman would be an optional display or export target.

The service would map stable assignment identity to a VTODO UID rather than treat the cache ID as an institutional identifier. Uploaded documents would be processed as data and never loaded as Python configuration. External edits would become new unverified input rather than silently replacing the verified service record.

Consequences and reversal

The selected boundary adds authentication, storage and export reconciliation work. It cannot constrain arbitrary writers of exported calendars; protection depends on the service retaining authority over its own record. Future acceptance must include non-owner, unverified-source, stale-version, retry, rejection and time-zone cases. No such implementation is claimed here.

For A3, rollback means discarding generated workspaces; upstream source is unchanged. A future deployment must be able to disable export and restore its prior service release without losing revision history. No production migration or rollback was executed.

Safe change boundary and review handoff

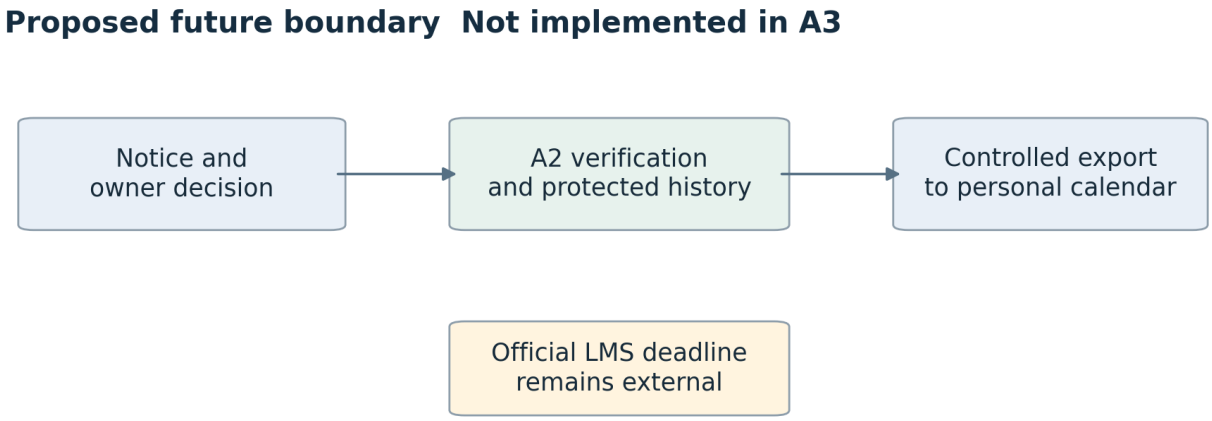


Figure 3. Proposed boundary for a later bounded implementation. The official LMS remains authoritative outside the personal verification service.

What the evidence supports

The recorded Linux environment can install its locked dependencies offline, build the frozen source, execute the upstream suite and complete the six isolated observations. The call-path reconstruction is supported by source and selected runtime calls. This establishes a reviewable investigation baseline, not production authorization, source authenticity or operational reliability.

Limits and unresolved questions

No live LMS/CalDAV, real participant evidence, interactive usability session, native Windows/WSL execution, DST study, process-crash recovery test or production deployment was performed. The file/cache interruption window remains an inference. The optional REPL and upstream exclusions remain unverified. A future implementation must establish a credible institutional source-authority procedure and confirm user value with authentic evidence.

Provenance and AI use

Primary source:
https://github.com/pimutils/todoman/tree/78e35788606a0add0c4f99ceb702c3ea0d0b1346

The unchanged source archive, licence, file hashes, runtime lock, raw output and decision record are included. ChatGPT/Codex assisted with discovery, execution, analysis and writing. AI-USE-AND-VERIFICATION.md records accepted/rejected advice and programmatic verification. No independent human or student rerun is claimed.